ShiftLink 원문→RAG 구축 과정

Oct 23, 2026 · @재영

요약

원문 58개 문서(2,459쪽)와 지식 카드 86장을 쪽 단위 700토큰 조각 3,950개로 나누고, embeddinggemma 300M으로 임베딩해 Jetson에서 질문마다 검색합니다. 검색 결과는 판정 모델이 걸러 낸 뒤 답변 모델에 근거로 넘어갑니다. 색인은 PC와 vast.ai GPU에서 만들고, Jetson은 만들어진 파일(조각 JSONL + 벡터 npy)을 읽기만 합니다.

구축은 다섯 단계를 차례로 거칩니다. 단계마다 결과 파일을 따로 남겨, 추출 오류와 검색 오류를 나눠서 볼 수 있게 했습니다.

1. **파싱·정제**: PDF를 쪽별 텍스트로 뽑고 머리말·목차·빈 쪽을 걸러 `pages.jsonl`을 만든다.
2. **청킹**: 쪽 안에서 절 제목·목록·표 단위로 묶어 700토큰 이하 조각을 만든다(`chunks_700.jsonl`). 카드는 1장 = 1조각이다.
3. **임베딩**: 조각마다 `[문서명 · 절 제목]`을 앞에 붙여 벡터로 바꾼다(`vectors_700__embeddinggemma_300m.npy`).
4. **검색**: 질문(+MES 이상 상태 줄)을 같은 모델로 임베딩하고, 설비 필터를 건 코사인 유사도로 상위 조각을 고른다. 근거 예산은 8조각·2,500토큰, 인계 기록 조각은 최대 1개다.
5. **판정·생성**: 판정 모델이 조각마다 관련 확률을 매겨 버리거나 남기고, 답변 모델이 남은 근거를 [S#]로 인용해 답한다.

항목	최종값 (C5, 2026-10-22)
원문	58개 문서, 2,459쪽(정상 2,233쪽)
색인 조각	3,950개 = 원문 3,824 + 카드 86 + 인계 기록 40
조각 크기	최대 700토큰, 쪽 경계를 넘지 않음, 같은 절 안에서 60토큰 겹침
임베딩	embeddinggemma 300M(0.6GB), 코사인, Jetson CPU에서 질문 임베딩 p50 0.77초
검색 방식	dense 단독 + 설비 필터(BM25·재순위화는 실험 후 제외)
근거 예산	8조각 / 2,500토큰, 인계 기록 최대 1조각

원문 소스

원문은 설비 제조사 매뉴얼·기술 문서(영어) 40개와 안전 지침·법령(대부분 한국어) 18개입니다. 여기에 현장 노하우를 요약한 지식 카드 86장과, 카드에 달린 과거 인계 기록을 함께 색인했습니다. 질문은 한국어인데 원문 조각의 73%(2,900/3,950)가 영어라는 점이 이후 모든 설계를 좌우했습니다.

폴더(설비)	내용	문서	쪽	원문 조각
safety	KOSHA 기술지침(E-14 감전 응급조치 등), 산업안전보건기준에 관한 규칙, OSHA 1910.147, HSE 인계 지침	18	436	902
CAU	공기압축기 매뉴얼, 압축공기 효율 자료	4	401	600
HPU	유압 유닛·펌프 매뉴얼	9	328	516
PDP	배전반·차단기 자료	6	359	509
GR	감속기 매뉴얼	8	326	483
CV	컨베이어 매뉴얼	4	342	459
shared	유압 정비·고장 분석, 압연기 베어링 사례	6	148	207
RT	롤러 컨베이어·코일카	3	119	148
원문 소계		58	2,459	3,824
지식 카드	설비 6종에 11~13장씩 + COMMON 12장, 카드 1장 = 1조각	—	—	86
인계 기록	카드에 달린 과거 사건 기록, 카드와 따로 자름	—	—	40
합계				**3,950**

수집과 배포에는 세 가지 제약을 두었습니다.

- **재배포 금지**: 제조사 PDF는 공개 저장소에 올리지 않는다. 저장소에는 코드와 기록만 두고, 프로젝트가 끝나면 RAG 부분을 지운다(10/13 결정).
- **외부 전송 금지**: 문서를 외부 모델 API로 보내지 않는다(9/30 결정). 대량 계산은 우리가 빌린 vast.ai GPU에서 한다.
- **원문 발췌 표시 허용**: 화면에 “문서명 p.N”과 함께 근거 문장을 그대로 보여 줄 수 있다(10/13 결정).

설비 폴더는 그대로 검색 필터가 됩니다. 작업자가 PDA에서 고른 설비의 폴더와 공통 폴더(shared·safety·COMMON)만 검색합니다.

파싱과 정제

2,459쪽 중 2,233쪽(91%)만 색인에 들어갔고, 카드가 인용한 원문 쪽의 90%(84/93)에서 카드의 핵심 단어가 추출 텍스트에 남아 있음을 확인했습니다. 파싱은 `shiftlink/rag2/parse.py` 가 맡고, 결과는 쪽 하나가 한 줄인 `pages.jsonl` 과 품질 보고서 `parse_report.json` 입니다. 파싱과 청킹을 분리한 것은 나중에 검색이 틀렸을 때 추출이 문제인지 자르기가 문제인지 따로 보기 위해서입니다.

추출기 선택

- **pypdf를 기본**으로 썼다. 한국어 PDF가 정상으로 나온다.
- **pdftotext는 암호화된 영어 PDF에만** 썼다. 한국어를 깨뜨리기 때문이다.
- 스캔 PDF는 없었다(빈 쪽 50% 초과 문서 0개). 그래서 OCR은 넣지 않았다.
- HTML 1개(FUCHS 윤활유 데이터시트)는 뒤로 미루고 제외했다. Markdown으로 정리된 안전 기준 4개는 그대로 읽었다.

정제 규칙

규칙	방법	왜
머리말·꼬리말 제거	숫자를 `#` 으로 바꾼 뒤 한 문서에서 3쪽 이상 반복되는 줄(3~120자)을 지운다. 5쪽 미만 문서는 건너뛴다	모든 조각에 같은 제품명·저작권 줄이 들어가 임베딩이 비슷해지는 것을 막는다
쪽 번호 제거	숫자만 있는 줄(`- 12 -` 등)을 지운다	검색에 무의미한 숫자가 들어가지 않게 한다
하이픈 줄넘김	`hydrau-\nlic` → `hydraulic`	단어가 두 토큰으로 갈리지 않게 한다
문장 중간 줄바꿈	마침표·`다`·`요` 뒤가 아닌 줄바꿈은 공백으로 합친다	PDF 줄 폭에서 생긴 문장 조각을 되살린다
빈 쪽	정제 후 80자 미만	그림·표지 쪽 190개를 뺀다
목차·색인 쪽	점선+쪽 번호로 끝나는 줄이 40% 초과, 또는 첫 줄이 Contents·Index·목차	목차 35쪽은 제목만 많아 검색 상위를 차지한다
중복 쪽	공백을 모두 뺀 텍스트의 SHA-1 앞 16자	같은 쪽이 근거 예산을 두 번 쓰지 않게 한다(1쪽)
깨진 문자	문서별 `\ufffd` 수를 세고 20개 초과 문서를 보고	사람이 확인할 문서 3개를 추렸다

쪽마다 문서명, 설비 폴더, 쪽 번호, 상태, 한글 비율, 해시를 함께 저장합니다. 한글 비율은 다음 단계에서 조각의 언어(`ko` / `en`)가 되고, 쪽 번호는 화면의 "문서명 p.N" 인용이 됩니다.

보존 대조(파싱 완료 기준)

"카드가 인용한 원문 쪽에 그 카드의 핵심 사실이 추출 텍스트로 남아 있다"를 완료 기준으로 잡았습니다. 카드의 출처 표기(예: `p.14 Troubleshooting`)에서 단어와 카드 본문의 숫자를 뽑아, 인용 쪽 ±1쪽 텍스트에 얼마나 들어 있는지 재었습니다.

- 출처 단어가 절반 이상 남은 쪽: **84/93(90%)**
- 카드 숫자가 절반 이상 남은 쪽: 53/91(카드 숫자는 단위 환산·요약을 거친 것이 많아 보조 지표로만 봤다)
- 둘 다 낮은 4건(K-1021 830CC p.6, K-1317·K-1318 차단기 지침)은 사람이 열어 확인할 목록으로 남겼다.

청킹 전략

조각 크기는 250·450·700토큰을 임베딩 모델 4개와 교차해 재고 700으로 정했습니다. 네 모델 모두 조각이 클수록 R@1·R@5가 올랐고, 쪽 경계를 넘지 않게 자르므로 700은 사실상 "한 쪽 = 한 조각"이 되어 인용과 정답 대조가 가장 단순해졌기 때문입니다. 청킹은 `shiftlink/rag2/chunk.py`가 맡습니다.

글자가 아니라 토큰으로 자른다

v1은 900자로 자랐는데, 영어에서는 약 200~250토큰, 한국어에서는 그 세 배 가까이여서 같은 "크기"가 언어마다 달랐습니다. 그래서 모델이 실제로 세는 토큰 수를 기준으로 바꿨습니다. 토크나이저를 매번 부르지 않도록, Ollama의 `prompt_eval_count`로 모델별 "글자당 토큰"을 재서 계수로 썼습니다.

임베딩 모델	한글 1자당 토큰	그 밖의 문자 1자당 토큰
bge-m3	0.75	0.24
embeddinggemma	0.92	0.23
qwen3-embedding	1.05	0.22
청킹에 쓴 값	**0.9**	**0.235**

세 모델의 가운데 값을 쓰되, 한국어는 넉넉하게 잡아 조각이 모델 입력 한도를 넘지 않게 했습니다.

분할 규칙

한 쪽을 먼저 뜻 단위 블록으로 나눈 뒤, 블록을 크기 한도까지 차례로 담습니다(greedy packing). 우선순위는 절 제목 → 문단 → 문장입니다.

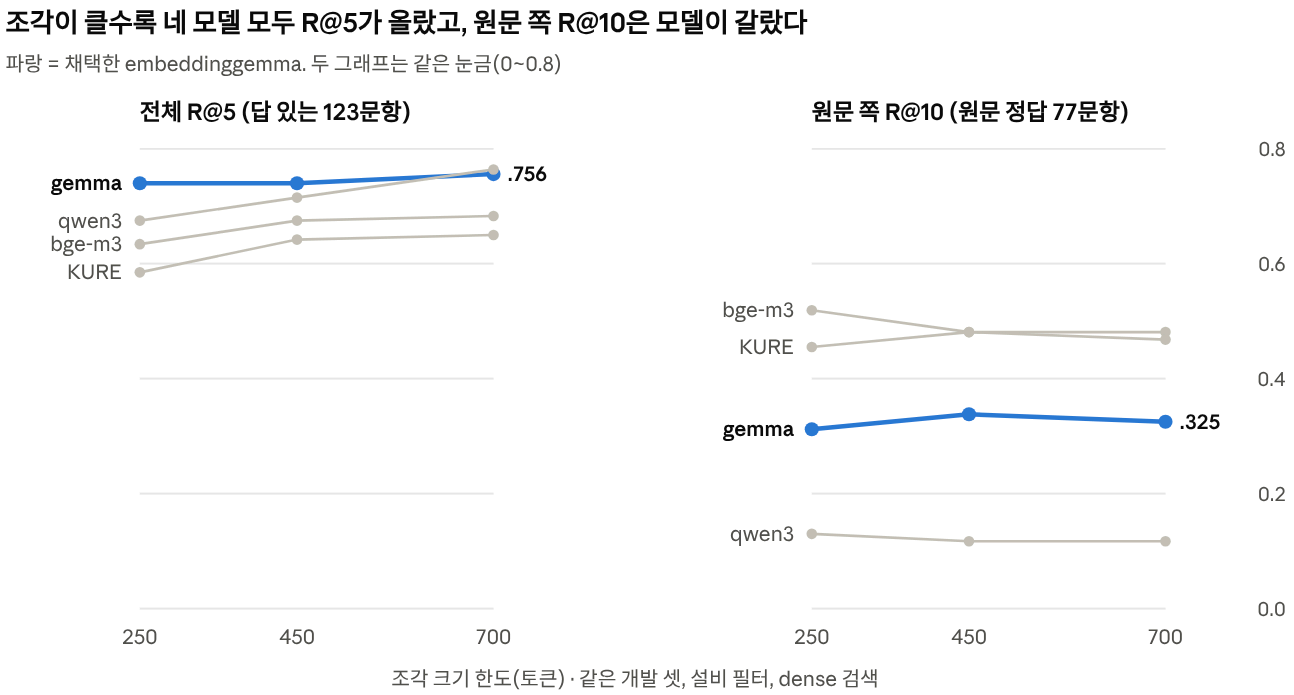
규칙	내용	왜
쪽 경계	조각은 한 쪽을 넘지 않는다	화면 인용 “문서명 p.N”과 정답 쪽 대조를 단순하게
절 제목	`제 N조`, `4.2 Troubleshooting`, `가.`, 대문자 제목 줄을 정규식으로 잡아 현재 절로 기억. 다음 쪽으로 이어진다	절이 바뀌면 조각도 끊는다. 제목은 임베딩 문맥에 쓴다
목록	`1.` · `a)` · `①` · `(1)` · `가.` 로 시작하는 줄은 한 묶음으로 유지	절차 순서가 두 조각으로 갈리지 않게
표	숫자가 든 행이 연속되면 표로 보고, 바로 위 머리줄을 각 행 앞에 `머리` 형태로 붙인다	행
겹침	같은 절 안에서만 앞 조각의 끝 문장을 60토큰까지 반복	경계에 걸린 조건-결과 문장을 살린다. 절이 바뀌면 겹치지 않는다
긴 문장	한도를 넘는 블록은 문장으로, 그래도 넘으면 글자 비율로 자른다	과대 조각 0개
작은 조각	40자 미만은 버린다	쪽 끝 자투리가 검색 상위를 차지하지 않게
카드	카드 1장 = 1조각(제목+증상+노하우+근거), 쪼개지 않는다	이미 검증된 요약이다. 1,000토큰을 넘는 것은 K-1315 하나뿐
인계 기록	카드에 달린 긴 사건 텍스트는 카드와 따로 자른다(`kind: event`)	카드 조각을 짧게 유지하고, 기록은 검색 상한을 따로 건다
문맥 머리	임베딩할 텍스트 앞에 `[문서명 · 절 제목]`, 카드는 `[카드 K-xxxx · 설비]`	“pressure relief valve”가 어느 설비·절인지 벡터에 담긴다

크기 후보와 실험

후보를 250·450·700으로 잡은 근거는 두 가지였습니다. 참고한 RAG 절차서의 권장 범위가 400~800토큰이고, 답변 모델(EXAONE 2.4B) 문맥이 4,096토큰이라 근거에 쓸 수 있는 예산이 처음에는 약 1,500토큰이었습니다. 250은 “작게 찾기”, 700은 “쪽 하나를 통째로” 극단이고, 450은 예산 안에 3~4개가 들어가는 중간값입니다.

크기 한도	조각 수
250토큰	6,429
450토큰	4,532
700토큰	3,951

세 크기를 같은 개발 셋(답 있는 123문항, 그중 원문 쪽 정답이 있는 77문항)과 설비 필터로, 임베딩 모델 4개에 대해 검색만 해서 비교했습니다. 정답은 “정답 카드” 또는 “정답 쪽 ±1쪽의 원문 조각”이 상위 k개에 들면 맞은 것으로 쳤습니다.



rag-v2-log.md 10/13 임베딩 격자 · 4개 모델 × 3개 크기

왼쪽은 크기가 커질수록 모든 선이 오르고, 오른쪽은 선이 거의 수평이라 높이(모델)만 다릅니다.

결정: 700토큰

- **상위권 재현율은 모든 모델에서 700 ≥ 450 > 250**이었다. 조각이 작으면 한 쪽의 내용이 여러 조각으로 흩어져 질문과 맞는 문맥이 줄어드는 것이 원인으로 보인다(가설, 따로 검증하지 않음).
- **원문 쪽을 찾는 비율은 크기보다 모델이 정했다.** bge-m3에서만 250이 약간 높았고(.519 대 .481), 나머지는 크기에 거의 무관했다.
- **700은 사실상 쪽 단위다.** 원문 조각 토큰의 중앙값은 197, p95는 659로, 대부분의 쪽이 한 조각에 들어간다. 화면 인용이 “문서 p.N” 하나로 끝나고, 근거 후보를 “서로 다른 쪽”으로 뽑을 수 있다.

- **조각 수가 가장 적다.** 3,951개는 6,429개의 61%로, Jetson에서 벡터 행렬을 메모리에 올리고 한 번에 곱하는 비용이 가장 작다.

사후 확인: 근거 예산을 바꾼 뒤에도 700

10/14에 근거 예산을 5조각·1,500토큰에서 8조각·2,500토큰으로 늘리면서(근거 안 정답 86 → 96/123) 크기를 다시 재습니다. 같은 예산에서 450은 82~97, 250은 86~100으로 700과 ±1~3문항 차이였고, 크기를 바꿀 이유가 없어 700을 유지했습니다. 예산을 늘렸을 때 실제로 들어가는 조각이 평균 4개였다는 점도 확인했습니다.

예외: 한 조각에 두 방법이 섞인 경우(10/22)

쪽 단위 조각의 약점이 안전 오답으로 나타난 적이 있습니다. KOSHA E-14(감전 응급조치) §8에서 반듯이 눕히는 흉부압박과 엎드린 자세의 니엘센법이 한 조각에 붙어 있어, 모델이 "엎드려 눕히고 흉부압박"이라고 답했습니다(NQ-D14).

- §8 조각 6개를 합친 뒤 방법 제목(`(가) 경구법`, `(다) 흉부압박`, `(라) 니엘센법` 등)에서 다시 잘라 5개로 만들었다. 색인은 3,951 → 3,950개가 됐다.
- "흉부압박 30회"와 "엎드려 눕히고"가 한 조각에 없을 것, 조각마다 방법 제목이 하나일 것, 700토큰 이하일 것을 assert로 확인했다.
- 다시 임베딩하기 전에 바뀌지 않은 조각 하나를 새로 임베딩해 저장된 벡터와 코사인 0.999 이상인지 보고, 바뀐 행만 교체했다.
- 결과: NQ-D14 오답이 사라졌다. 같은 방법을 다른 안전 지침으로 넓히는 것은 아직 하지 않았다.

임베딩 모델 선택

임베딩은 embeddinggemma 300M을 골랐습니다. 원문 쪽을 찾는 능력은 bge-m3가 가장 좋았지만, 실제로 답을 생성해 보니 gemma 쪽이 정답 근거를 더 많이 근거 목록에 넣고(86 대 77/123) 더 많이 인용했으며(77 대 71), 모델 크기는 절반(0.6GB 대 1.2GB)이었기 때문입니다.

키워드 검색(BM25)을 주 검색에서 뺀 이유

한국어 질문으로 영어 원문 쪽을 찾는 비율이 BM25는 2.6%였습니다. 단어가 걸치지 않으니 당연한 결과입니다. 두 방식을 섯으면(RRF) 상위권은 조금 좋아지지만, BM25가 한국어 카드를 끌어올려 원문 쪽을 밀어냈습니다.

방식 (450토큰, 설비 필터)	R@1	R@5	R@10	원문 쪽 R@10
BM25	.358	.593	.740	.026

방식 (450토큰, 설비 필터)	R@1	R@5	R@10	원문 쪽 R@10
bge-m3 dense	.439	.675	.805	.481
bge-m3 + BM25 (RRF, 같은 비중)	.472	.724	.789	.325
bge-m3 + BM25 (RRF, BM25 비중 0.25)	.504	.691	—	.377

BM25 비중을 낮춰도 원문 쪽 R@10은 dense 단독(.481)으로 돌아오지 않았습니다. RAG v2의 목적이 “카드에 없는 원문 근거”를 찾는 것이었으므로 주 검색은 dense로 했습니다.

후보 4개 비교(700토큰, dense, 설비 필터)

모델	크기	PC CPU 임베딩(조각당)	R@1	R@5	R@10	원문 쪽 R@10
embeddinggemma 300M	0.6GB	0.51초	**.537**	.756	.837	.325
qwen3-embedding 0.6B	0.6GB	2.12초	.480	**.764**	**.894**	.117
bge-m3 567M	1.2GB	1.08초	.472	.683	.805	**.481**
KURE-v1(bge-m3 한국어 미세조정)	1.2GB	1.12초	.415	.650	.780	.468

- **모델마다 강점이 갈렸다.** qwen3·gemma는 한국어 카드를 잘 찾고 영어 원문을 못 찾았다. bge-m3는 원문을 가장 잘 찾았다.
- **한국어 미세조정 모델(KURE)은 원본 bge-m3보다 전반적으로 낮았다.** 우리 문제는 “한국어 이해”가 아니라 “한→영 교차 검색”이었다.
- 문서 전체 임베딩은 PC CPU로는 느려 vast.ai RTX 3090에서 했고, 질문은 PC의 같은 버전 Ollama(0.40.3)로 만들었다. 문서와 질문에 같은 모델·같은 접두사를 쓴다는 규칙을 지켰다.

카드와 원문을 따로 찾는 레인 방식

카드는 dense+BM25로, 원문은 dense만으로 따로 찾아 “카드 1 · 원문 2” 순서로 교차해 보았습니다. bge-m3 레인(BM25 비중 0.25)이 원문 쪽 R@10 .532로 가장 높았고 R@1도 .537이었습니다. 카드는 gemma, 원문은 bge-m3를 쓰는 2-모델 레인은 단일 bge-m3 레인보다 나아지지 않아 기각했습니다.

최종 결정은 생성 결과로

검색 지표만으로는 A(bge-m3 레인)와 B(gemma dense)가 갈려 두 구성으로 실제 답을 생성해 비교했습니다(201문항, EXAONE 3.5 2.4B, temperature 0).

지표	A bge-m3 레인	B gemma dense
정답 근거가 근거 목록에 듦 (/123)	77	**86**
정답 근거를 인용 (/123)	71	**77**
근거에 없는 숫자를 쓴 답	**3**	5
프롬프트 최대 토큰	3,227	**2,489**
모델 크기	1.2GB	**0.6GB**

B를 골랐습니다. A의 레인은 원문 쪽을 더 많이 가져오지만, 카드가 세 자리 중 한 자리만 얻는 구조라 전체 정답 근거는 오히려 적었습니다(원인은 추정). Jetson(8GB)에는 답변 모델과 판정 모델이 함께 올라가야 하므로 0.6GB 차이도 컸습니다. Jetson에서는 임베딩을 CPU(`num_gpu: 0`)로 돌려 GPU를 언어 모델에 남겨 두었고, 질문 임베딩은 p50 0.77초·p95 1.01초였습니다.

쓰지 않기로 한 것

후보	결과	결정
재순위화(bge-reranker-v2-m3, 상위 30)	근거 안 정답이 예산 4개 모두에서 오히려 감소(예: 8/2500에서 96 → 82). PC CPU로는 50분을 넘겨도 끝나지 않음	기각
유사도 임계값으로 “답 없음” 가르기	1위 유사도 중앙값이 답 있음 .653, 답 없음 .623으로 거의 같음	거절은 판정 모델이 맡음
벡터 DB(ChromaDB 1.0.20)	numpy와 상위 10개 99.9% 일치, 질의 14.0ms 대 3.9ms. Jetson(ARM) 설치는 미확인	numpy 행렬(`.npy`)로 충분
2-모델 혼합(카드 gemma + 원문 bge-m3, 10조각/3,000토큰)	근거 안 정답 102로 가장 높음	Jetson 메모리와 학습·추론 근거 설정 불일치로 보류

색인 구성

색인은 파일 두 개입니다. 조각 3,950개를 한 줄씩 담은 `chunks_700.jsonl`(10.3MB)과, 같은 순서의 768차원 벡터 행렬 `vectors_700__embeddinggemma_300m.npy`(3,950 × 768 float32, 12.1MB)입니다. 벡터는 L2 정규화해 저장하므로 내적이 곧 코사인 유사도이고, 검색은 행렬 곱셈 한 번입니다. 별도

벡터 DB 서버가 없어 Jetson에 띄울 프로세스가 하나 줄었습니다.

조각 한 줄의 필드

필드	예	쓰임
`chunk_id`	`HY29-0022-UK_p14_c02`, `card_K-1004`, `event_K-1104_c00`	인용 연결, 부분 교체
`text`	본문	답변 모델에 근거로 넣는 텍스트
`embed_text`	`[문서명 · 절 제목] 본문`	임베딩에만 쓰는 텍스트
`tokens`	197	근거 예산(2,500토큰) 계산
`metadata.source`, `page`, `section`	`Troubleshooting.pdf`, 14	화면 출처 "문서명 p.N", 같은 쪽 중복 제거
`metadata.folder`	HPU, safety, COMMON	설비 필터
`metadata.kind`	source / card / event	인계 기록 상한, 화면 표기
`metadata.lang`	en / ko	한↔영 검색 성능 분석
`metadata.safety`, `source_class`	true, safety_guide	안전 지침 구분
`metadata.content_hash`	SHA-1 앞 16자	변경 감지, 중복 제거

권한 필드(`allowed_roles`)는 두지 않았습니다. 장비 한 대에서 한 부서가 쓰는 온디바이스 시스템이라 권한을 나눌 대상이 없었습니다.

빌드와 배포

1. PC에서 `python -m shiftlink.rag2.parse` → `pages.jsonl`, `parse_report.json`
2. PC에서 `python -m shiftlink.rag2.chunk --sizes 700` → `chunks_700.jsonl` (크기·중복 ID·절 없음 개수를 출력해 확인)

3. vast.ai GPU에서 `python -m shiftlink.rag2.embed --model embeddinggemma:300m` → 벡터 `.npy` + `manifest.json`(모델, 차원, 거리 함수, 접두사, 소요 시간). 20묶음마다 `.partial.npy`를 저장해 끊겨도 이어서 돌린다.
4. 두 파일을 Jetson의 앱 데이터 폴더로 복사하고, MES 서비스가 시작할 때 메모리에 올린다.

문서와 질문은 반드시 같은 모델·같은 접두사를 씁니다. embeddinggemma는 문서 앞에 `title: none | text: `, 질문 앞에 `task: search result | query: `를 붙입니다. 모델을 바꾸면 전체를 다시 임베딩합니다.

부분 교체

일부 조각만 바꿀 때는 전체를 다시 만들지 않고 해당 행만 교체합니다(10/22 E-14 분할). 교체 전에 바뀌지 않은 조각 하나를 지금 환경에서 다시 임베딩해, 저장된 벡터와 코사인 0.999 이상인지 확인합니다. 다른 GPU·다른 Ollama에서 만든 벡터가 섞이는 것을 막기 위해서입니다. 실제로 10/14에 다른 GPU로 근거를 다시 붙였을 때 725쌍 중 19쌍의 근거가 달라진 적이 있습니다.

질의 시점 검색

질문 하나는 Jetson에서 "검색어 만들기 → 설비 필터 검색 → 근거 고르기 → 판정 → 답변" 순서로 처리됩니다. 보류 셋 43문항(10/16, 당시 답변 모델 ck106) 기준 p50 7.75초, p95 9.77초이었고, 그중 임베딩은 p50 0.77초·판정 3.83초·생성 3.31초입니다. 코드는 `shiftlink/rag2/s30h.py`입니다.

1. **대상 설비 정하기**: PDA가 보낸 설비와 다른 설비 코드(GR-02 등)가 질문에 있으면 그 설비를 기준으로 바꾸고 선택 설비의 MES 값은 쓰지 않는다.
2. **관측값 없음 확인**: 질문이 묻는 신호에 유효한 MES 값이 없으면 검색하지 않고 코드가 "현장 계기로 직접 확인하세요"로 답한다.
3. **검색어 만들기**: 질문 뒤에 MES의 낮음·높음 상태를 한 줄씩 붙인다(예: `클램프 압력 90 bar (정상 100–120 bar 미만)`). 정상이면 질문만 남는다. 이 줄은 검색에만 쓰고 판정·답변 모델에는 원래 질문을 넘긴다.
4. **검색**: 검색어를 CPU에서 임베딩하고, 설비 종류 폴더와 공통 폴더(shared·safety·COMMON) 조각만 남겨 코사인 상위 30개를 뽑는다.
5. **근거 고르기**: 순위대로 담되 같은 문서·같은 쪽은 하나만, 최대 8조각·2,500토큰, 인계 기록은 1개까지. 다른 설비의 인계 기록은 빼고 빈자리를 채우지 않는다.
6. **판정**: 판정 모델(exaone-sft-judge)이 조각마다 0~3점 분포를 내고, P(2점 이상)으로 경로를 정한다(아래 표).
7. **답변과 검사**: 답변 모델(ck176)이 남은 근거를 [S#]로 인용해 답한다. 코드가 근거에 없는 숫자·잘못된 인용을 검사하고, 다른 설비를 말하는 문장을 잘라내며, 답 앞에 MES 현재값 줄을, 답 끝에 "근거: 문서 p.N"을, 화면 위에 설비의 안전 카드를 붙인다.

판정 결과	경로	동작
모든 조각이 P < 0.3	drop_all	답하지 않음(“해당 지식 없음”). 정상 범위를 묻는 질문은 MES 범위로 코드가 답함
남은 조각 중 최댓값 ≥ 0.9	confident	답변을 강제(출력 앞에 `"insufficient_evidence": false`를 미리 채움)
그 사이	model	답변 모델이 판단. 거절 확률 p_insuff ≥ 0.3528이면 거절

0.3·0.9·0.3528은 10/15에 결과를 보기 전에 등록한 값이고, 이후 바꾸지 않았습니다. 보류 셋에서는 confident 18 · model 11 · drop_all 14문항으로 나뉘었습니다.

검색어에 MES 상태를 붙인 근거

“압력이 이상해”처럼 짧은 질문은 어느 신호가 어느 쪽으로 벗어났는지 담지 못합니다. 10/20 실험(12상황 × 4문형, 답 있는 48행, 2반복)에서 상태 줄을 붙이자 정답 카드가 검색 후보 8개에 드는 행이 40 → 48, 정답 카드 인용이 32 → 41(McNemar p 0.0039)이 됐습니다. 다만 사람 판정의 “충분한 답” 순증이 −1로 사전등록 관문 하나를 통과하지 못했고, 채택은 이 점을 알고 한 사용자 결정이었습니다(PR #312).

평가

검색은 “정답 근거가 답변 모델에 넘어가는 근거 목록에 들었는가”로 평가했고, 개발 셋에서는 86/123에서 99/123까지 올렸습니다. 그러나 처음 보는 질문(보류 셋)에서는 17/35에 그쳤고, 최종 모델의 오답 절반 이상도 검색 실패였습니다. RAG 파이프라인에서 가장 큰 남은 손실은 검색입니다.

지표

지표	정의	왜 썼나
R@k	정답 카드 또는 정답 쪽 ±1쪽의 원문 조각이 상위 k개에 있는 답 있는 문항 비율	검색만 따로 쟀다(LLM 없음)
원문 쪽 R@10	원문 쪽 정답이 있는 77문항에서, 원문 조각으로 맞힌 비율	“카드에 없는 원문 근거”라는 v2의 목적을 따로 본다
근거 안 정답(근거 상한)	예산 안에 들어간 근거에 정답 조각이 있는 문항 수	답변 모델이 맞히는 것의 상한이다
오거절 원인	답 있는 문항을 거절했을 때 근거에 정답이 있었는가	모델 문제와 검색 문제를 가른다

단계별 결과

시점	평가	결과
10/13	검색 격자(모델 4 × 크기 3 × 방식)	gemma 700 R@5 .756, R@10 .837
10/13	생성 비교 A 대 B	근거 안 정답 77 대 86/123 → B 채택
10/14	근거 예산·인계 기록 상한	86 → 99/123(구성 C)
10/16	보류 셋 43문항(Jetson, 1회)	근거 안 정답 17/35. 오거절 10건 중 9건이 검색 실패
10/20	MES 상태 줄을 검색어에	정답 카드가 후보에 듦 40 → 48/48
10/22	봉인 셋 S1(C5)	오답 30건 중 17건이 검색이 정답 근거를 못 가져온 경우

시행착오에서 얻은 것

- **검색 지표가 좋다고 답이 좋아지지는 않았다.** 원문 쪽 R@10이 가장 높은 A가 생성에서는 B보다 정답 근거·인용이 적었다. 최종 선택은 생성 결과로 했다.
- **검색 설정을 바꾸면 모델도 다시 학습해야 했다.** 구성 B로 학습한 모델을 구성 C 근거로 돌리자 근거 안 정답은 86 → 99로 늘었는데 답한 문항은 78 → 71로 줄었다. 구성 C로 학습 데이터를 다시 만들어 학습한 뒤(보강 106) 답함 99·정답 인용 68로 올랐다.
- **인계 기록 조각은 상한이 필요했다.** 제한이 없으면 기록이 근거 자리를 차지해, 1개로 제한하자 근거 안 정답이 96 → 99가 됐다. 다른 설비의 기록을 빼되 빈자리를 다음 순위로 채우지 않은 것도 실험 결과다(채우면 충분한 답이 줄었다).

- **자동 지표는 품질을 낮게 쟀다.** “답했지만 정답 근거를 인용하지 않음”으로 분류된 답 중 판정기 19/25, SFT 15/21을 사람이 맞음 또는 일부 맞음으로 판정했다. 주요 결정에는 블라인드 사람 채점을 함께 썼다.
- **개발 셋 수치는 낙관적이었다.** 같은 201문항에서 설정을 고르고 쟀기 때문이다. 그래서 설정을 고르기 전에 기준을 커밋하는 사전등록, 두 절반 교차 검사, 한 번만 돌리는 보류 셋을 두었고, 보류 셋에서 검색의 한계가 드러났다.